

UNIVERSIDADE FEDERAL DE JUIZ DE FORA
CYGNO - Grupo UFJF

Atividade: Redes Neurais com PyTorch (MLP e CNN)

Introdução e Objetivos

Na atividade anterior, construímos um classificador capaz de diferenciar eventos de Recuo Eletrônico (ER) e Recuo Nuclear (NR) a partir das variáveis (*features*) extraídas de cada evento. Nesta atividade, vamos dar o próximo passo: usar **redes neurais**, implementadas com a biblioteca **PyTorch**.

Vamos trabalhar com dois tipos de rede:

- **MLP** (*Multilayer Perceptron*): uma rede neural "densa" que recebe as mesmas *features* usadas na atividade anterior. É a continuação direta da Atividade 2.
- **CNN** (*Convolutional Neural Network*): uma rede que recebe diretamente a **imagem** do evento e aprende sozinha quais padrões são importantes para a classificação.

Ao final, você terá:

- Familiaridade com os elementos básicos do PyTorch: tensores, uso da GPU, `nn.Module`, `Dataset` e `DataLoader`.
- Compreensão do ciclo de treinamento de uma rede neural (*forward*, função de perda, *backpropagation* e atualização dos pesos).
- Compreensão de conceitos como conjunto de validação, *overfitting* e *early stopping*.
- Uma MLP treinada com as *features* e uma CNN treinada diretamente com as imagens dos eventos, comparadas nos mesmos eventos de teste.
- Experiência prática com os principais parâmetros de uma rede neural (camadas, neurônios, ativação, otimizador, taxa de aprendizado, *BatchNorm*, *Dropout*, etc.).

Ferramentas recomendadas: Google Colab com GPU (necessária para a parte da CNN).

Parte 0: Configuração do Ambiente

Treinar uma CNN com milhares de imagens exige muito processamento. Em uma CPU comum, cada época de treinamento levaria vários minutos; em uma GPU, leva alguns segundos. Por isso, vamos usar o **Google Colab**, que oferece uma GPU gratuitamente.

Arquivos da Atividade

A pasta `dados_atividade` contém três arquivos:

- `dataset_2classes.csv`: tabela com as *features* de cada evento (ER e NR com energias de 3, 5, 10, 15, 20 e 25 keV). É o mesmo dataset usado na Atividade 2.
- `cnn_dataset_2classes.h5`: as imagens de cada evento (arquivo no formato HDF5, com ∼420 MB). As imagens já estão pré-processadas: a imagem completa do sensor tem 2304 × 2304 pixels, e cada evento foi recortado em uma janela de 224 × 224 pixels centrada no cluster.
- `split_indices_ER_NR.json`: indica quais linhas do `.csv` pertencem ao conjunto de **treino** e quais pertencem ao conjunto de **teste**.

Preparando o Colab

- **Tarefa:** No seu Google Drive, crie a pasta `CYGNO/atividade_3/` e envie para ela a pasta `dados_atividade` inteira.
- **Tarefa:** Crie um novo notebook no Colab e ative a GPU em *Ambiente de execução → Alterar o tipo de ambiente de execução → GPU T4*.
- **Tarefa:** Monte o Google Drive e defina os caminhos dos arquivos.

  Dica: o Colab já vem com `torch`, `h5py`, `scikit-learn`, `pandas` e `matplotlib`. Para ver um resumo das camadas das redes, instale o `torchinfo` com `%pip install torchinfo`.

Passo a passo para acessar os dados pelo Drive:

```
import os
import shutil
from google.colab import drive

drive.mount('/content/drive')

PASTA_DADOS = '/content/drive/MyDrive/CYGNO/atividade_3/dados_atividade'

CAMINHO_CSV   = os.path.join(PASTA_DADOS, 'dataset_2classes.csv')
CAMINHO_SPLIT = os.path.join(PASTA_DADOS, 'split_indices_ER_NR.json')
CAMINHO_H5    = os.path.join(PASTA_DADOS, 'cnn_dataset_2classes.h5')
```

```
12
13 # Ler um arquivo grande direto do Drive é lento: copiamos o .h5 para o disco local do Colab
14 shutil.copy(CAMINHO_H5, '/content/cnn_dataset_2classes.h5')
15 CAMINHO_H5 = '/content/cnn_dataset_2classes.h5'
```

Importações e Reprodutibilidade

Todas as bibliotecas usadas na atividade são importadas no início do notebook. Redes neurais começam com pesos **aleatórios** e os dados de treino são embaralhados a cada época; por isso, fixamos a **semente** (*seed*) dos geradores de números aleatórios, para que os resultados sejam (quase) os mesmos toda vez que o código for executado.

```
1  # %pip install torchinfo      # rode uma vez no Colab, se o torchinfo não estiver instalado
2
3  import json
4  import random
5  import time
6
7  import numpy as np
8  import pandas as pd
9  import h5py
10 import matplotlib.pyplot as plt
11
12 import torch
13 import torch.nn as nn
14 import torch.nn.functional as F
15 import torch.optim as optim
16 from torch.utils.data import Dataset, TensorDataset, DataLoader
17 from torchinfo import summary
18 from tqdm.auto import tqdm
19
20 from sklearn.model_selection import train_test_split
21 from sklearn.preprocessing import StandardScaler
22 from sklearn.metrics import (classification_report, confusion_matrix, ConfusionMatrixDisplay,
23                              roc_curve, roc_auc_score)
24
25
26 def definir_seed(seed=42):
27     random.seed(seed)
28     np.random.seed(seed)
29     torch.manual_seed(seed)
30     torch.cuda.manual_seed_all(seed)
31     print(f"Seeds definidas para {seed}")
32
33
34 SEED = 42
35 definir_seed(SEED)
```

Dica: chame `definir_seed(SEED)` *também logo antes de criar cada modelo, para que os pesos iniciais sejam sempre os mesmos.*

Parte 1: Visualizando o Dataset

Antes de treinar qualquer modelo, vamos olhar os dados. Cada linha do `.csv` é um evento, e a imagem desse evento está no arquivo `.h5`, em `train_images` ou `test_images`. A ligação entre os dois arquivos é feita pelo `split_indices_ER_NR.json`: a imagem `train_images[i]` corresponde à linha `split['train'][i]` do `.csv` (e o mesmo vale para o teste).

O código abaixo cria, para cada linha do `.csv`, duas informações: em qual conjunto do `.h5` está a imagem (`conjunto_h5`) e em qual posição (`indice_h5`).

```
df = pd.read_csv(CAMINHO_CSV)

with open(CAMINHO_SPLIT, 'r') as f:
    split = json.load(f)

# Para cada linha do .csv: em qual conjunto do .h5 está a imagem ('train' ou 'test') e em qual posição
conjunto_h5 = np.empty(len(df), dtype=object)
indice_h5 = np.full(len(df), -1)

conjunto_h5[split['train']] = 'train'
indice_h5[split['train']] = np.arange(len(split['train']))
conjunto_h5[split['test']] = 'test'
indice_h5[split['test']] = np.arange(len(split['test']))
```

Com isso, a função `plotar_evento` escolhe um evento pelo tipo (`'ER'` ou `'NR'`), pela energia e pelo número do evento (`n`) — ou, se preferir, pelo `run` e pelo `event` — e lê apenas a imagem desse evento do `.h5`. O parâmetro `zoom` mostra só a região central da imagem, onde está o cluster, e a função também imprime algumas *features* do evento.

```
def plotar_evento(tipo='ER', energia=10, n=0, run=None, event=None, zoom=None, ax=None):
    # Seleciona os eventos do tipo e da energia escolhidos
    selecao = df[(df['particle_type'] == tipo) & (df['energy_kevee'] == energia)]
    if run is not None and event is not None:     # escolha pelo run e pelo event
        selecao = selecao[(selecao['run'] == run) & (selecao['event'] == event)]
        n = 0
    linha = selecao.index[n]                      # número da linha no .csv
    evento = df.loc[linha]

    # Lê apenas a imagem desse evento no .h5
    with h5py.File(CAMINHO_H5, 'r') as f:
        img = f[f'{conjunto_h5[linha]}_images'][indice_h5[linha], :, :, 0]

    if zoom is not None:                          # recorta a região central de lado 'zoom'
        c = img.shape[0] // 2
        img = img[c - zoom // 2: c + zoom // 2, c - zoom // 2: c + zoom // 2]

    if ax is None:
        fig, ax = plt.subplots(figsize=(5, 5))
    im = ax.imshow(img, cmap='viridis')
    plt.colorbar(im, ax=ax, fraction=0.046)
    ax.set_title(f"{tipo} - {energia} keV - run {evento['run']}, ev {evento['event']}", fontsize=10)

    print(f"Linha {linha} | sc_nhits = {evento['sc_nhits']} | Width = {evento['Width']:.1f} | "
```

```
            f"LAPA = {evento['LAPA']:.1f} | Energy_calibrated = {evento['Energy_calibrated']:.2f} keV")
    return img

plotar_evento('NR', energia=15, n=0)                          # o primeiro NR de 15 keV
plt.show()
plotar_evento('ER', energia=25, run=1, event=10, zoom=60)      # escolhendo pelo run e pelo event
plt.show()
```

Usando a função, montamos uma figura com um evento ER e um evento NR para cada energia (como na Figura 1) e outra com vários eventos do mesmo tipo e da mesma energia, para ver a variabilidade entre eles:

```
energias = [3, 5, 10, 15, 20, 25]

fig, ax = plt.subplots(2, len(energias), figsize=(19, 7))
for j, energia in enumerate(energias):
    for i, tipo in enumerate(['ER', 'NR']):
        plotar_evento(tipo, energia, n=0, zoom=60, ax=ax[i, j])
        ax[i, j].axis('off')
plt.tight_layout()
plt.show()

# Variabilidade: 6 eventos ER (em cima) e 6 eventos NR (embaixo) de 5 keV
fig, ax = plt.subplots(2, 6, figsize=(19, 7))
for i, tipo in enumerate(['ER', 'NR']):
    for n in range(6):
        plotar_evento(tipo, energia=5, n=n, zoom=40, ax=ax[i, n])
        ax[i, n].axis('off')
plt.tight_layout()
plt.show()
```

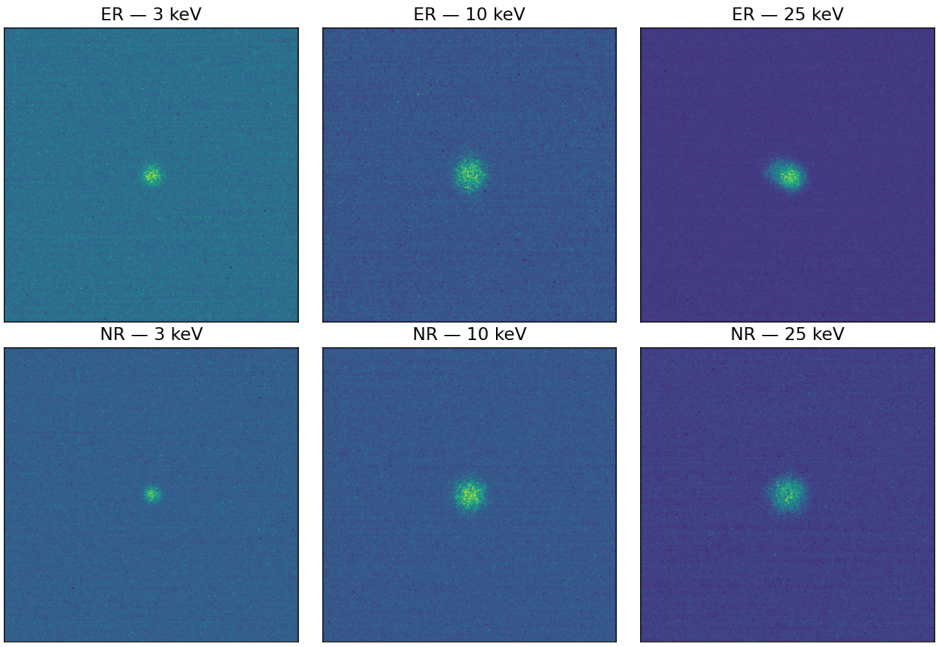


Figura 1: Exemplos de eventos ER e NR em diferentes energias (224 × 224 pixels).

- **Tarefa:** Execute o código acima e explore outros eventos, mudando o tipo, a energia,

o `n` e o *zoom*.

- **Pergunta:** Quais diferenças visuais entre ER e NR você observa? Como elas mudam com a energia? Em quais energias você acha que será mais difícil separar as duas classes?

Parte 2: Primeiros Passos com PyTorch

O PyTorch é uma das bibliotecas mais usadas para construir e treinar redes neurais. O seu objeto básico é o **tensor**: um *array* multidimensional, muito parecido com o `np.array` do NumPy, mas que pode ser processado na **GPU** e que sabe calcular **gradientes** automaticamente.

Versão, CUDA e Dispositivo

As GPUs da NVIDIA são programadas através do **CUDA**, e o PyTorch usa o CUDA "por baixo dos panos" sempre que um tensor ou um modelo está na GPU. No PyTorch, o **dispositivo** (*device*) indica onde um tensor ou um modelo está: na CPU (`'cpu'`) ou na GPU (`'cuda'`).

```
import torch

print("Versão do PyTorch:", torch.__version__)
print("CUDA disponível?", torch.cuda.is_available())
if torch.cuda.is_available():
    print("Versão do CUDA:", torch.version.cuda)
    print("Número de GPUs:", torch.cuda.device_count())
    print("Nome da GPU:", torch.cuda.get_device_name(0))
    print(f"Memória da GPU: {torch.cuda.get_device_properties(0).total_memory / 1e9:.1f} GB")

# Usa a GPU se ela estiver disponível; caso contrário, usa a CPU
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print("Dispositivo usado:", DEVICE)
```

- **Tarefa:** Verifique a versão do PyTorch, a versão do CUDA, o nome e a memória da GPU disponível no Colab. Execute também `!nvidia-smi` em uma célula e compare as informações.
- **Tarefa:** Troque o ambiente de execução do Colab para CPU, execute o código novamente e observe o que muda. Depois, volte para a GPU.
- **Pergunta:** O que é o CUDA e por que ele é necessário para usar a GPU no PyTorch?
- **Pergunta:** Por que uma GPU é muito mais rápida do que uma CPU para treinar redes neurais?

Tensores

```python
# Criando tensores
a = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
b = torch.ones(2, 2)
print(a + b, a @ b)                    # soma e produto matricial
print(a.shape, a.dtype, a.device)      # formato, tipo e dispositivo

# Mudando o formato
c = torch.arange(12)
print(c.reshape(3, 4).shape, c.unsqueeze(0).shape)

# Conversão NumPy <-> PyTorch
t = torch.from_numpy(np.arange(6).reshape(2, 3))
print(t.numpy())

# Movendo para a GPU e de volta para a CPU
a_gpu = a.to(DEVICE)
a_cpu = a_gpu.cpu()
```

- **Tarefa:** Crie um tensor aleatório (`torch.randn`) com o formato de um lote de 64 imagens da CNN: $(64, 1, 224, 224)$. Quantos elementos ele tem e quanta memória ocupa?

  Dica: use os métodos `.numel()` *e* `.element_size()`*.*

- **Tarefa:** Tente somar um tensor que está na CPU com um tensor que está na GPU. Leia a mensagem de erro e explique o que ela significa.

- **Pergunta:** Por que é preciso usar `.cpu()` antes de converter um tensor da GPU para NumPy com `.numpy()`?

CPU × GPU

Vamos medir na prática a diferença de velocidade, multiplicando matrizes $n \times n$ na CPU e na GPU:

```python
import time

def medir_tempo(device, n=4000, repeticoes=10):
    A = torch.randn(n, n, device=device)
    B = torch.randn(n, n, device=device)
    C = A @ B                          # aquecimento (a primeira chamada é mais lenta)
    if device.type == 'cuda':
        torch.cuda.synchronize()       # espera a GPU terminar
    inicio = time.perf_counter()
    for _ in range(repeticoes):
        C = A @ B
    if device.type == 'cuda':
        torch.cuda.synchronize()
    return (time.perf_counter() - inicio) / repeticoes

print(f"CPU: {medir_tempo(torch.device('cpu')):.4f} s")
```

```
print(f"GPU: {medir_tempo(torch.device('cuda')):.4f} s")
```

- **Tarefa:** Meça o tempo na CPU e na GPU para $n = 100, 500, 1000, 2000$ e 4000. Plote o tempo em função de n para os dois dispositivos (use escala logarítmica nos dois eixos).
- **Pergunta:** A vantagem da GPU é a mesma para matrizes pequenas e grandes? Por quê?
- **Pergunta:** Por que é necessário usar `torch.cuda.synchronize()` para medir o tempo na GPU?

Gradientes Automáticos

Treinar uma rede neural significa ajustar seus pesos para minimizar uma função de perda (*loss*). Para isso, precisamos das derivadas da perda em relação a cada peso, que o PyTorch calcula automaticamente (*autograd*) com o método `.backward()`.

```
# y = w^2 + 3w  ->  dy/dw = 2w + 3
w = torch.tensor(2.0, requires_grad=True)   # "quero a derivada em relação a w"
y = w**2 + 3*w
y.backward()                                # calcula dy/dw
print(w.grad)                               # tensor(7.)
```

- **Tarefa:** Use o *autograd* para calcular a derivada de $y = w\,\mathrm{sin}(w)$ em $w = \pi$. Confira o resultado calculando a derivada à mão.

Neurônios e Camadas

Um neurônio calcula uma soma ponderada das entradas e aplica uma função de ativação: $a = \sigma(\mathbf{w}\cdot\mathbf{x}+b)$. Uma camada `nn.Linear` é um conjunto de neurônios, $\mathbf{a} = \sigma(W\mathbf{x}+\mathbf{b})$. A ativação mais usada é a ReLU: $\sigma(z) = \max(0, z)$.

```
import torch.nn as nn

camada = nn.Linear(in_features=4, out_features=3)   # 4 entradas -> 3 neurônios
x = torch.randn(5, 4)                               # um lote (batch) com 5 exemplos de 4 features
saida = torch.relu(camada(x))
print(saida.shape, camada.weight.shape, camada.bias.shape)
```

- **Pergunta:** Para uma camada `nn.Linear(4, 3)`, quais são os formatos da matriz de pesos W e do vetor $\mathbf{b}$? Quantos parâmetros essa camada tem?

Parte 3: MLP com as Features do Evento

Nesta parte, vamos treinar uma rede neural com as *features* do evento, como na Atividade 2.

Preparação dos Dados

Usaremos a divisão treino/teste do arquivo `split_indices_ER_NR.json` tanto para a MLP quanto para a CNN. Assim, as duas redes serão avaliadas nos mesmos eventos, e a comparação será justa.

Nem toda coluna do `.csv` deve entrar no modelo. Removemos: os rótulos (`particle_type`, `particle_type_original`); as informações verdadeiras da simulação (`energy_kevee`, `truth_integral`); os identificadores (`run`, `event`); a posição do cluster no sensor (`xmin`, `xmax`, `ymin`, `ymax`, `xmean`, `ymean`); e algumas variáveis redundantes com outras já presentes (`sc_integral`, `SDSD_weighted`, `dE_dX_uncalib` e `dE_dA_uncalib`).

```
idx_treino_completo = np.array(split['train'])   # linhas do .csv usadas para treino
idx_teste = np.array(split['test'])               # linhas do .csv usadas para teste
print(f"Eventos de treino: {len(idx_treino_completo)} | Eventos de teste: {len(idx_teste)}")
print(pd.crosstab(df['particle_type'], df['energy_kevee']))   # eventos por classe e energia

colunas_excluidas = ['particle_type', 'particle_type_original',
                     'energy_kevee', 'truth_integral',
                     'run', 'event',
                     'xmin', 'xmax', 'ymin', 'ymax', 'xmean', 'ymean',
                     'sc_integral', 'SDSD_weighted', 'dE_dX_uncalib', 'dE_dA_uncalib']
features = [c for c in df.columns if c not in colunas_excluidas]
print(f"{len(features)} features usadas: {features}")

# Matriz de features (X) e vetor alvo (y): ER -> 0, NR -> 1
X_todos = df[features].values.astype(np.float32)
y_todos = (df['particle_type'] == 'NR').astype(np.int64).values

X_treino_completo, y_treino_completo = X_todos[idx_treino_completo], y_todos[idx_treino_completo]
X_teste, y_teste = X_todos[idx_teste], y_todos[idx_teste]
```

- **Pergunta:** Por que as colunas `energy_kevee` e `truth_integral` não devem ser usadas como *features*? E por que a posição do cluster no sensor também foi removida?

Conjunto de Validação

Em redes neurais, além do treino e do teste, usamos um terceiro conjunto: a **validação**. Ele é usado **durante** o treinamento para acompanhar se a rede está generalizando bem e para decidir quando parar o treinamento. O conjunto de **teste** só deve ser usado **uma vez**, no final.

Separamos 25% dos eventos de treino para validação. O `stratify` mantém a proporção de ER e NR nos dois conjuntos. Repare que separamos os **índices** (`pos_treino` e `pos_val`), e não os dados: eles serão reutilizados para separar as imagens da CNN.

```
from sklearn.model_selection import train_test_split

pos_treino, pos_val = train_test_split(
    np.arange(len(y_treino_completo)),
    test_size=0.25,
    random_state=42,
    stratify=y_treino_completo
)

X_treino, y_treino = X_treino_completo[pos_treino], y_treino_completo[pos_treino]
X_val, y_val = X_treino_completo[pos_val], y_treino_completo[pos_val]
print(f"Treino: {len(y_treino)} | Validação: {len(y_val)} | Teste: {len(y_teste)}")
```

Padronização das Features

As *features* têm escalas muito diferentes (por exemplo, `sc_nhits` $\sim 10^3$ e `rho` $\sim 10^{-2}$). Por isso, cada *feature* é padronizada: $x' = (x - \mu)/\sigma$.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_treino_pad = scaler.fit_transform(X_treino)   # calcula média/desvio e transforma
X_val_pad = scaler.transform(X_val)             # apenas transforma
X_teste_pad = scaler.transform(X_teste)         # apenas transforma
```

- **Pergunta:** Na Atividade 2, a *Random Forest* foi treinada sem padronização. Por que a rede neural precisa dela?
- **Pergunta:** Por que a média e o desvio-padrão são calculados **apenas** com o conjunto de treino?

Dataset e DataLoader

No PyTorch, os dados são entregues à rede em **lotes** (*batches*) por um `DataLoader`. A cada passada completa pelos dados de treino damos o nome de **época**. O `TensorDataset` junta as *features* e os rótulos; no treino, os dados são embaralhados a cada época (`shuffle=True`), e na validação e no teste não.

```
from torch.utils.data import TensorDataset, DataLoader

BATCH_SIZE_MLP = 128

ds_treino_mlp = TensorDataset(torch.from_numpy(X_treino_pad.astype(np.float32)), torch.from_numpy(
    y_treino))
ds_val_mlp = TensorDataset(torch.from_numpy(X_val_pad.astype(np.float32)), torch.from_numpy(y_val))
```

```
ds_teste_mlp = TensorDataset(torch.from_numpy(X_teste_pad.astype(np.float32)), torch.from_numpy(y_teste))

loader_treino_mlp = DataLoader(ds_treino_mlp, batch_size=BATCH_SIZE_MLP, shuffle=True)
loader_val_mlp = DataLoader(ds_val_mlp, batch_size=BATCH_SIZE_MLP, shuffle=False)
loader_teste_mlp = DataLoader(ds_teste_mlp, batch_size=BATCH_SIZE_MLP, shuffle=False)
```

Definindo a MLP

Toda rede no PyTorch é uma classe que herda de `nn.Module`. As camadas são definidas no `__init__` e o caminho da entrada até a saída é definido no `forward`. A saída tem **2 neurônios**, um para cada classe. A camada `Dropout` "desliga" neurônios aleatoriamente durante o treino, o que ajuda a reduzir o *overfitting*.

```
from torchinfo import summary

class MLP(nn.Module):
    def __init__(self, n_features, n_classes=2):
        super().__init__()
        self.rede = nn.Sequential(
            nn.Linear(n_features, 64), nn.ReLU(), nn.Dropout(0.2),
            nn.Linear(64, 32),         nn.ReLU(), nn.Dropout(0.2),
            nn.Linear(32, n_classes)
        )

    def forward(self, x):
        return self.rede(x)

definir_seed(SEED)           # pesos iniciais reprodutíveis
modelo_mlp = MLP(n_features=len(features)).to(DEVICE)
summary(modelo_mlp, input_size=(BATCH_SIZE_MLP, len(features)))
```

- **Pergunta:** Pela saída do `summary`, quantos parâmetros treináveis a MLP tem? Quais camadas têm parâmetros e quais não têm?

Treinamento

O núcleo de qualquer treinamento em PyTorch é o laço abaixo, que percorre uma época de treino, lote por lote. A função de perda `CrossEntropyLoss` é a padrão para classificação, e o otimizador `Adam` é o responsável por atualizar os pesos.

```
import torch.optim as optim

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(modelo_mlp.parameters(), lr=1e-3)

modelo_mlp.train()      # modo de treino (ativa o Dropout)
for entradas, rotulos in loader_treino_mlp:
    entradas, rotulos = entradas.to(DEVICE), rotulos.to(DEVICE)

    optimizer.zero_grad()               # 1. zera os gradientes do lote anterior
```

```
    saidas = modelo_mlp(entradas)       # 2. forward: a rede faz a previsão
    loss = criterion(saidas, rotulos)   # 3. calcula o erro (perda)
    loss.backward()                     # 4. backpropagation: calcula os gradientes
    optimizer.step()                    # 5. atualiza os pesos
```

Depois de cada época de treino, a rede é avaliada no conjunto de validação. Nessa etapa não há atualização de pesos, então desligamos o cálculo de gradientes. As saídas da rede (*logits*) são convertidas em probabilidades com a função *softmax*; a coluna 1 é a probabilidade de o evento ser NR, que é usada para calcular a AUC.

```
import torch.nn.functional as F
from sklearn.metrics import roc_auc_score

modelo_mlp.eval()       # modo de avaliação (desativa o Dropout)
soma_loss, todos_rotulos, todas_probs = 0.0, [], []
with torch.no_grad():   # não precisamos de gradientes na validação
    for entradas, rotulos in loader_val_mlp:
        entradas, rotulos = entradas.to(DEVICE), rotulos.to(DEVICE)
        saidas = modelo_mlp(entradas)
        soma_loss += criterion(saidas, rotulos).item()

        probs = F.softmax(saidas, dim=1)[:, 1]     # probabilidade de NR
        todos_rotulos.append(rotulos.cpu().numpy())
        todas_probs.append(probs.cpu().numpy())

val_loss = soma_loss / len(loader_val_mlp)
val_auc = roc_auc_score(np.concatenate(todos_rotulos), np.concatenate(todas_probs))
```

Para acompanhar o treinamento, é importante mostrar o progresso. A biblioteca `tqdm` cria uma barra de progresso para os lotes de cada época, e, ao final de cada época, imprimimos um resumo com as métricas de treino e de validação:

```
from tqdm.auto import tqdm

barra = tqdm(loader_treino_mlp, desc='  [Treino]', leave=False)
for entradas, rotulos in barra:
    ...                                                 # passos 1 a 5 do laço de treino
    barra.set_postfix({'loss': f"{loss.item():.4f}"})     # mostra a perda do lote na barra

# Ao final de cada época
print(f"Época {epoca+1:03d}/{epocas} - {tempo:4.1f}s | "
      f"[Treino] Loss: {train_loss:.4f}, Acc: {train_acc:.2f}%, AUC: {train_auc:.4f} | "
      f"[Val] Loss: {val_loss:.4f}, Acc: {val_acc:.2f}%, AUC: {val_auc:.4f} | {status}")
```

Exemplo da linha impressa a cada época (o `status` indica se o modelo foi salvo ou quantas épocas se passaram sem melhora, para o *early stopping*):

```
Época 010/200 -  0.3s | [Treino] Loss: 0.3837, Acc: 78.70%, AUC: 0.8911 | [Val] Loss: 0.3728, Acc:
    78.48%, AUC: 0.8965 | Salvo
```

- **Tarefa:** Junte os trechos acima em uma função `treinar_modelo`, que receba o modelo, os `DataLoader`s de treino e de validação, a função de perda e o otimizador, e que repita o treino e a validação por várias épocas, mostre o progresso com o `tqdm`,

imprima o resumo de cada época e guarde, a cada época, a perda, a acurácia e a AUC de treino e de validação. Essa mesma função será usada depois para treinar a CNN.

Dica: a classe prevista é a de maior saída: `saidas.argmax(dim=1)`.

- **Tarefa:** Inclua na função o ***early stopping*** e o salvamento do **melhor modelo**: sempre que a perda de validação atingir um novo mínimo, salve os pesos; se ela não melhorar por um certo número de épocas seguidas (paciência), interrompa o treinamento.

  Dica: os pesos são salvos com `torch.save(model.state_dict(), caminho)`.

- **Tarefa:** Inclua também um *scheduler*, que reduz a taxa de aprendizado quando a perda de validação estagna.

  Dica: o scheduler é criado uma vez e chamado ao final de cada época:

```
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=5)
...
scheduler.step(val_loss)    # ao final de cada época
```

- **Tarefa:** Treine a MLP e plote as curvas de perda, acurácia e AUC de treino e de validação ao longo das épocas.

  Sugestão de parâmetros: `batch_size=128`, `lr=1e-3`, *até 200 épocas, paciência de 15 épocas.*

- **Pergunta:** O que aconteceria se esquecêssemos o `optimizer.zero_grad()`? E qual é a função do `model.eval()` na validação?

- **Pergunta:** Observando as curvas, a rede apresentou *overfitting*? Como isso aparece nos gráficos?

Avaliação no Conjunto de Teste

- **Tarefa:** Carregue os pesos do melhor modelo salvo e calcule a probabilidade de NR de cada evento do conjunto de teste. Guarde essas probabilidades em `prob_mlp`: elas serão usadas nas Partes 5 e 7.

  Dica: `modelo_mlp.load_state_dict(torch.load('melhor_mlp.pth'))`.

- **Tarefa:** Gere o `classification_report`, a `confusion_matrix` e calcule a AUC.

- **Pergunta:** Compare o resultado da MLP com o do classificador da Atividade 2. A rede neural trouxe ganho?

Parte 4: CNN com as Imagens dos Eventos

Agora a rede **não** recebe *features* prontas: ela recebe a imagem do evento e aprende, sozinha, filtros que destacam os padrões úteis para separar ER de NR.

Carregando as Imagens

O arquivo `.h5` contém os conjuntos `train_images` $(7936, 224, 224, 1)$, `train_labels`, `test_images` $(1984, 224, 224, 1)$ e `test_labels`. Desta vez, carregamos todas as imagens na memória e removemos o último eixo (o canal), ficando com arrays de formato $(N, 224, 224)$.

Em seguida, conferimos que os rótulos do `.h5` são iguais aos do `.csv` e separamos treino e validação com **os mesmos índices** da MLP (`pos_treino` e `pos_val`). Por fim, normalizamos as imagens com a média e o desvio-padrão dos pixels do **treino**, $x' = (x - \mu)/\sigma$. Como as imagens ocupam cerca de 2 GB de memória RAM, a normalização é feita "no lugar" (`-=` e `/=`), sem criar cópias dos arrays.

```
with h5py.File(CAMINHO_H5, 'r') as f:
    imgs_treino_completo = f['train_images'][:, :, :, 0]   # (7936, 224, 224)
    rot_treino_completo = f['train_labels'][:].astype(np.int64)
    imgs_teste = f['test_images'][:, :, :, 0]              # (1984, 224, 224)
    rot_teste = f['test_labels'][:].astype(np.int64)

# Os rótulos do .h5 são os mesmos do .csv?
print(np.array_equal(rot_treino_completo, y_treino_completo), np.array_equal(rot_teste, y_teste))

# Treino e validação com os MESMOS índices da MLP
imgs_treino, rot_treino = imgs_treino_completo[pos_treino], rot_treino_completo[pos_treino]
imgs_val, rot_val = imgs_treino_completo[pos_val], rot_treino_completo[pos_val]
del imgs_treino_completo                                    # libera memória RAM

# Normalização com a média e o desvio dos pixels do treino, feita "no lugar"
media_pixels = imgs_treino.mean()
desvio_pixels = imgs_treino.std()
for imgs in [imgs_treino, imgs_val, imgs_teste]:
    imgs -= media_pixels
    imgs /= desvio_pixels
```

Um Dataset Personalizado

Para as imagens, criamos a nossa própria classe `Dataset`. Ela precisa de dois métodos: `__len__`, que retorna o número de exemplos, e `__getitem__`, que retorna **um** exemplo (imagem, rótulo). As camadas convolucionais esperam imagens no formato (canais, altura, largura); como as nossas imagens têm um único canal, o `unsqueeze(0)` transforma cada imagem de $(224, 224)$ em $(1, 224, 224)$. Com o `Dataset` pronto, criamos os `DataLoaders` de treino, validação e teste, da mesma forma que na MLP.

```
from torch.utils.data import Dataset
```

```
2
3  class DatasetImagens(Dataset):
4      def __init__(self, imagens, rotulos):
5          self.imagens = imagens      # array (N, 224, 224) já normalizado
6          self.rotulos = rotulos      # array (N,)
7
8      def __len__(self):
9          return len(self.imagens)
10
11     def __getitem__(self, idx):
12         img = torch.from_numpy(self.imagens[idx]).unsqueeze(0)   # (224, 224) -> (1, 224, 224)
13         rotulo = torch.tensor(self.rotulos[idx], dtype=torch.long)
14         return img, rotulo
15
16 BATCH_SIZE_CNN = 64
17
18 loader_treino_cnn = DataLoader(DatasetImagens(imgs_treino, rot_treino), batch_size=BATCH_SIZE_CNN,
       shuffle=True)
19 loader_val_cnn = DataLoader(DatasetImagens(imgs_val, rot_val), batch_size=BATCH_SIZE_CNN, shuffle=False
       )
20 loader_teste_cnn = DataLoader(DatasetImagens(imgs_teste, rot_teste), batch_size=BATCH_SIZE_CNN, shuffle
       =False)
```

Arquitetura da CNN

A rede abaixo é um **exemplo** de CNN, baseado em uma arquitetura usada no grupo. Não existe uma arquitetura "certa": o número de blocos, de filtros, de camadas do classificador, etc., são escolhas de projeto, que serão exploradas na Parte 6.

A CNN tem duas partes. A primeira é o **extrator de características**, formado por 6 blocos com:

- `Conv2d(kernel_size=3, padding=1)`: filtros 3×3 que "deslizam" pela imagem. Cada filtro gera um novo mapa (canal). Com `padding=1`, a altura e a largura da imagem são mantidas;
- `ReLU`: ativação não linear;
- `MaxPool2d(2)`: reduz a imagem pela metade, mantendo o maior valor de cada janela 2×2.

A cada bloco, o número de filtros dobra ($16 \to 32 \to \cdots \to 512$) e o tamanho da imagem cai pela metade ($224 \to 112 \to \cdots \to 3$). A segunda parte é o **classificador**: uma MLP que recebe a saída dos blocos convolucionais transformada em um vetor (`Flatten`).

```
1  class CNN(nn.Module):
2      def __init__(self, n_classes=2, tamanho_entrada=224):
3          super().__init__()
4          self.features = nn.Sequential(
5              nn.Conv2d(1, 16, kernel_size=3, padding=1),    nn.ReLU(), nn.MaxPool2d(2),
6              nn.Conv2d(16, 32, kernel_size=3, padding=1),   nn.ReLU(), nn.MaxPool2d(2),
7              nn.Conv2d(32, 64, kernel_size=3, padding=1),   nn.ReLU(), nn.MaxPool2d(2),
```

```
            nn.Conv2d(64, 128, kernel_size=3, padding=1),  nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(128, 256, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(256, 512, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
        )

        # Descobre automaticamente quantos valores saem dos blocos convolucionais
        with torch.no_grad():
            entrada_teste = torch.zeros(1, 1, tamanho_entrada, tamanho_entrada)
            n_saidas_conv = self.features(entrada_teste).view(1, -1).size(1)

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(n_saidas_conv, 128), nn.ReLU(),
            nn.Linear(128, 64),            nn.ReLU(),
            nn.Linear(64, 32),             nn.ReLU(),
            nn.Linear(32, 16),             nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(16, n_classes)
        )

    def forward(self, x):
        return self.classifier(self.features(x))

definir_seed(SEED)
modelo_cnn = CNN().to(DEVICE)
summary(modelo_cnn, input_size=(1, 1, 224, 224))
```

- **Pergunta:** Compare o número total de parâmetros da CNN com o da MLP e com o número de imagens de treino. O que isso sugere sobre o risco de *overfitting*?

Treinamento e Avaliação

- **Tarefa:** Treine a CNN na GPU com a **mesma** função `treinar_modelo` usada para a MLP e anote o tempo médio por época.

  Sugestão de parâmetros: `batch_size=64`, `lr=1e-4`, *até 100 épocas, paciência de 10 épocas.*

- **Tarefa:** Plote as curvas de treinamento. Compare o comportamento das curvas de treino e validação com o da MLP.

- **Tarefa:** Carregue o melhor modelo e calcule a probabilidade de NR de cada evento de teste. Guarde essas probabilidades em `prob_cnn`. Depois, gere o relatório de classificação e a matriz de confusão e calcule a AUC.

Parte 5: Comparação entre MLP e CNN

Como as duas redes foram avaliadas nos mesmos eventos de teste, podemos compará-las diretamente.

Para estudar como o desempenho depende da energia, vamos usar a **energia calibrada** (`Energy_calibrated`), que é a energia **reconstruída** do evento, e não a energia verdadeira da simulação (`energy_kevee`), que não existe nos dados reais. Os eventos de teste são agrupados em bins de 2–6, 6–10, 10–14, 14–18 e 18–22 keV:

```
energia_calib_teste = df['Energy_calibrated'].values[idx_teste]
bin_energia_teste = pd.cut(energia_calib_teste, [2, 6, 10, 14, 18, 22])   # bin de cada evento de teste
print(pd.crosstab(bin_energia_teste, y_teste))                            # eventos ER (0) e NR (1) por bin
```

- **Tarefa:** Plote, no mesmo gráfico, as curvas ROC da MLP e da CNN, com a AUC de cada uma na legenda.
- **Tarefa:** Calcule a AUC de cada rede separadamente em cada bin de energia calibrada e plote a AUC em função da energia.
- **Pergunta:** Qual rede teve o melhor desempenho? A diferença entre elas é grande?
- **Pergunta:** Em quais energias a separação entre ER e NR é mais difícil? Relacione com o que você observou nas imagens da Parte 1.
- **Pergunta:** Quantos eventos de cada classe existem em cada bin? O que isso implica para a confiabilidade da AUC calculada em cada bin?
- **Pergunta:** A AUC calculada com todas as energias juntas representa bem o desempenho em baixas energias? Por quê?
- **Pergunta:** Quais são as vantagens e desvantagens de usar a imagem do evento (CNN) em vez das *features* reconstruídas (MLP)? Pense em desempenho, custo computacional e interpretabilidade. Lembre que a CNN não vê a imagem completa do sensor (2304×2304), e sim um recorte de 224×224 já centrado no cluster.

Parte 6: Experimentos com os Parâmetros das Redes

Agora é a sua vez de explorar. Uma rede neural tem muitos parâmetros de projeto (os **hiperparâmetros**): número de camadas, número de neurônios, função de ativação, otimizador, taxa de aprendizado, etc. Nesta parte, você vai variar esses parâmetros e observar o efeito de cada um. Mude **uma coisa de cada vez** em relação à configuração original, para saber qual mudança causou cada efeito.

Experimentos com a MLP

A MLP treina em poucos segundos, então é possível testar muitas configurações.

- **Tarefa (camadas):** Varie o número de camadas escondidas: `[64]`, `[64, 32]`, `[128, 64, 32]` e `[256, 128, 64, 32]`.
- **Tarefa (neurônios):** Varie o número de neurônios: `[8, 4]`, `[64, 32]` e `[256, 128]`.
- **Tarefa (ativação):** Troque a ReLU por `nn.Tanh`, `nn.Sigmoid`, `nn.LeakyReLU` e `nn.GELU`.
- **Tarefa (*BatchNorm*):** Treine a rede com e sem *BatchNorm* (`nn.BatchNorm1d` depois de cada `nn.Linear` escondida).
- **Tarefa (*Dropout*):** Treine com *Dropout* de 0 (sem *Dropout*), 0,2 e 0,5.
- **Tarefa (otimizador):** Troque o `Adam` por `SGD`, por `SGD` com *momentum* de 0,9, por `RMSprop` e por `AdamW` com *weight decay* de 10^{-2}.

  Dica: o SGD sem momentum costuma precisar de uma taxa de aprendizado maior. Teste `lr=1e-3` e `lr=1e-2`.
- **Tarefa (taxa de aprendizado):** Teste `lr` = 10^{-2}, 10^{-3} e 10^{-4} com o `Adam`.
- **Tarefa (tamanho do lote):** Teste `batch_size` = 16, 128 e 1024.
- **Tarefa (pré-processamento e *scheduler*):** Treine sem a padronização das *features* e, separadamente, sem o *scheduler*.
- **Tarefa:** Para cada grupo de experimentos, plote juntas as curvas de perda de validação ao longo das épocas.
- **Pergunta:** Redes maiores (mais camadas e mais neurônios) foram sempre melhores? Por quê?
- **Pergunta:** Como o otimizador, a taxa de aprendizado e o tamanho do lote afetaram a **velocidade** de convergência e o resultado final?
- **Pergunta:** Qual foi o efeito do *BatchNorm*? E o de retirar a padronização das *features*?

Experimentos com a CNN

Cada treino da CNN leva alguns minutos, então escolha com cuidado o que testar.

- **Tarefa:** Escolha pelo menos três das modificações abaixo e treine a CNN novamente:
  - adicionar *BatchNorm* (`nn.BatchNorm2d`) depois de cada `Conv2d` (e, depois, retirá-lo de novo);
  - usar menos blocos convolucionais (por exemplo, 4 ou 5);
  - usar metade do número de filtros em cada bloco;
  - usar *kernels* 5×5;
  - mudar o *Dropout* do classificador;
  - trocar o otimizador (por exemplo, SGD com *momentum* e `lr=1e-3`);
  - mudar a taxa de aprendizado e o tamanho do lote.
- **Pergunta:** Como cada modificação afetou o número de parâmetros, o tempo por época e a AUC? Por que usar menos blocos convolucionais pode **aumentar** o número de parâmetros da rede?
- **Tarefa:** Registre todos os experimentos em uma tabela como a Tabela 1.

Tabela 1: Exemplo de tabela para registrar os experimentos.

Rede	**Modificação**	**Parâmetros**	**Épocas**	**AUC (teste)**	**Observações**
MLP	Original				
MLP	Com *BatchNorm*				
MLP	Otimizador SGD				
CNN	Original				
CNN	Com *BatchNorm*				

Parte 7: Os Modelos na Prática

Para fechar, vamos usar as redes treinadas nas Partes 3 e 4 para classificar **um evento de cada vez**, escolhido da mesma forma que na Parte 1. Usamos apenas eventos do conjunto de **teste**, que as redes nunca viram. Como o conjunto de teste está na mesma ordem de `split['test']`, a posição `indice_h5[linha]` de um evento de teste é também a sua posição em `X_teste_pad`, `imgs_teste`, `prob_mlp` e `prob_cnn`.

Classificando um Evento

Para aplicar uma rede a um único evento, a entrada precisa ter o formato de um lote com um só exemplo: $(1, n_{features})$ para a MLP e $(1, 1, 224, 224)$ para a CNN. A função abaixo seleciona o evento, aplica as duas redes e mostra a imagem ao lado da probabilidade de NR dada por cada uma (em verde se acertou e em vermelho se errou).

```
def classificar_evento(tipo='ER', energia=10, n=0, zoom=60):
    # Seleciona apenas eventos do conjunto de teste
    selecao = df[(df['particle_type'] == tipo) & (df['energy_kevee'] == energia) & (conjunto_h5 == '
    test')]
    linha = selecao.index[n]
    pos = indice_h5[linha]                    # posição do evento no conjunto de teste

    # MLP: features padronizadas, formato (1, n_features)
    x_mlp = torch.from_numpy(X_teste_pad[pos:pos + 1].astype(np.float32)).to(DEVICE)
    # CNN: imagem normalizada, formato (1, 1, 224, 224)
    x_cnn = torch.from_numpy(imgs_teste[pos]).reshape(1, 1, 224, 224).to(DEVICE)

    modelo_mlp.eval()
    modelo_cnn.eval()
    with torch.no_grad():
        p_mlp = F.softmax(modelo_mlp(x_mlp), dim=1)[0, 1].item()
        p_cnn = F.softmax(modelo_cnn(x_cnn), dim=1)[0, 1].item()

    # Imagem do evento (à esquerda) e probabilidade de NR de cada rede (à direita)
    fig, ax = plt.subplots(1, 2, figsize=(11, 4.5))
    plotar_evento(tipo, energia, run=df.loc[linha, 'run'], event=df.loc[linha, 'event'], zoom=zoom, ax=
    ax[0])

    probs = [p_mlp, p_cnn]
    previsoes = ['NR' if p > 0.5 else 'ER' for p in probs]
    cores = ['tab:green' if prev == tipo else 'tab:red' for prev in previsoes]
    ax[1].barh(['MLP', 'CNN'], probs, color=cores)
    ax[1].axvline(0.5, color='k', ls='--', label='Limiar (0,5)')
    for i, (p, prev) in enumerate(zip(probs, previsoes)):
        ax[1].text(min(p + 0.02, 0.80), i, f'{p:.3f} -> {prev}', va='center')
    ax[1].set_xlim(0, 1)
    ax[1].set_xlabel('Probabilidade de ser NR')
    ax[1].set_title(f'Classe verdadeira: {tipo}')
    ax[1].legend(loc='lower right')
    plt.tight_layout()
    plt.show()

classificar_evento('NR', energia=20, n=0)
classificar_evento('ER', energia=3, n=0)
```

Galeria de Acertos e Erros da CNN

A função abaixo sorteia vários eventos de teste em um intervalo de energia calibrada e mostra, no título de cada imagem, a classe verdadeira e a classe prevista pela CNN.

```
def galeria_cnn(e_min, e_max, n_eventos=10, zoom=50, seed=0):
    # Sorteia eventos de teste com energia calibrada entre e_min e e_max
    gerador = np.random.default_rng(seed)
```

```
    candidatos = np.where((energia_calib_teste >= e_min) & (energia_calib_teste < e_max))[0]
    posicoes = gerador.choice(candidatos, size=n_eventos, replace=False)

    fig, ax = plt.subplots(2, n_eventos // 2, figsize=(3 * n_eventos // 2, 6.8))
    with h5py.File(CAMINHO_H5, 'r') as f:
        for eixo, pos in zip(ax.flat, posicoes):
            img = f['test_images'][pos, :, :, 0]          # imagem original (sem normalização)
            c = img.shape[0] // 2
            eixo.imshow(img[c - zoom // 2: c + zoom // 2, c - zoom // 2: c + zoom // 2], cmap='viridis'
    )
            verdadeiro = ['ER', 'NR'][y_teste[pos]]
            previsto = ['ER', 'NR'][int(prob_cnn[pos] > 0.5)]
            eixo.set_title(f'Verd.: {verdadeiro} | CNN: {previsto}\nP(NR) = {prob_cnn[pos]:.2f}',
                           color='green' if verdadeiro == previsto else 'red')
            eixo.axis('off')
    plt.suptitle(f'CNN - energia calibrada entre {e_min} e {e_max} keV')
    plt.tight_layout()
    plt.show()

galeria_cnn(2, 6)
galeria_cnn(14, 18)
```

Os Erros Mais "Confiantes" da CNN

Por fim, procuramos os eventos em que a CNN errou com probabilidade mais próxima de 0 ou de 1, ou seja, os erros em que ela estava mais "confiante":

```
erros = np.where((prob_cnn > 0.5).astype(int) != y_teste)[0]
confianca = np.abs(prob_cnn[erros] - 0.5)              # distância até o limiar
piores = erros[np.argsort(confianca)[::-1][:8]]       # os 8 erros mais confiantes

fig, ax = plt.subplots(2, 4, figsize=(13, 7))
with h5py.File(CAMINHO_H5, 'r') as f:
    for eixo, pos in zip(ax.flat, piores):
        img = f['test_images'][pos, :, :, 0]
        eixo.imshow(img[87:137, 87:137], cmap='viridis')   # região central 50 x 50
        eixo.set_title(f"{energia_calib_teste[pos]:.1f} keV | Verd.: {['ER', 'NR'][y_teste[pos]]}\n"
                       f"P(NR) = {prob_cnn[pos]:.2f}", color='red')
        eixo.axis('off')
plt.suptitle('Erros mais confiantes da CNN')
plt.tight_layout()
plt.show()

# Quantos erros a CNN cometeu em cada bin de energia calibrada
print(pd.Series(bin_energia_teste[erros]).value_counts(sort=False))
```

- **Tarefa:** Execute o código acima para eventos ER e NR de baixa e de alta energia, e para diferentes intervalos de energia na galeria.
- **Pergunta:** As duas redes concordam entre si? Como se comportam as probabilidades em baixa e em alta energia?
- **Pergunta:** Em que energias estão os erros mais confiantes? Olhando as imagens, dá para entender por que a rede errou?

Parte 8: Apresentação dos Resultados

- **Tarefa:** Prepare uma apresentação de slides com os resultados da MLP e da CNN: as arquiteturas usadas, as curvas de treinamento, as métricas no conjunto de teste, a comparação entre as duas redes (curvas ROC e AUC por energia), os experimentos com os hiperparâmetros e exemplos das redes aplicadas a eventos individuais.